

LARA: Learned Adaptive Recombined Alignments

A Certifying Foundation Model for Petri Net Trace Alignment

Alessandro Berti Wil M.P. van der Aalst

Process and Data Science (PADS), RWTH Aachen University

ICPM 2027

The Problem

- **Alignments** explain, event by event, how a trace deviates from a process model
- Optimal alignments need **exact shortest-path search** (A^*): often the **bottleneck** on real discovered models
- Purely **learned** aligners are fast, but may be **illegal** or **silently suboptimal**

The tension

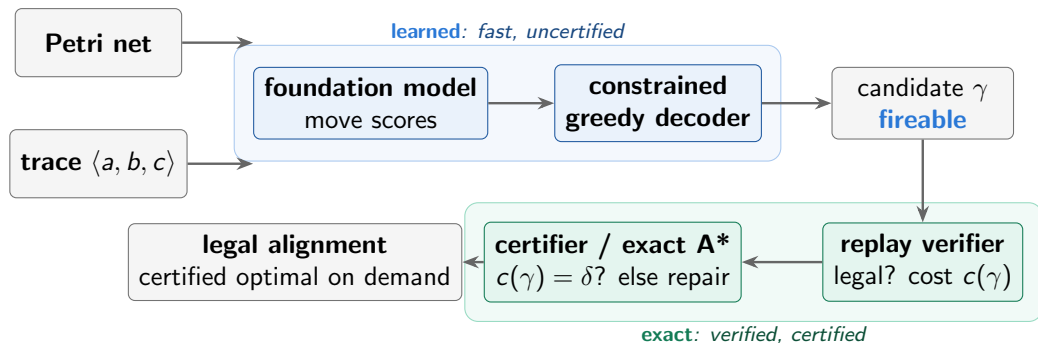
trust (exact, slow) vs. **tractability** (learned, fast)

Fast alignments that are never less trustworthy

A small neural **foundation model**, trained **once** on synthetic data, scores alignment moves for **any Petri net and any log**; exact methods **verify, certify, or repair** every answer.

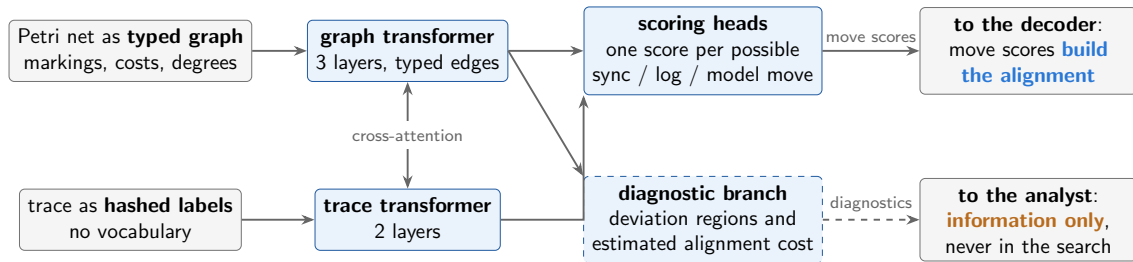
- One invariant: **neural output is never trusted as a certificate**
- Every answer is a **legal alignment with a known cost**; optionally a **certified optimal** one
- **Vocabulary-free**: no retraining, no configuration per log

The Approach at a Glance



Degrades from **certified optimal** to a **safe upper bound**, **never to wrong**.

Architecture of the Foundation Model



- Two outputs, two roles: the **move scores** continue down the pipeline and steer the decoder; the **diagnostics** (where the trace deviates, at what estimated cost) go to the **analyst only**
- The diagnostics carry **no guarantee**, so they never influence the search or the result
- Small model: about **3M parameters**, runs on **CPU**; all scores are **trace-dependent**

Training Loss: Imitating the Exact Aligner

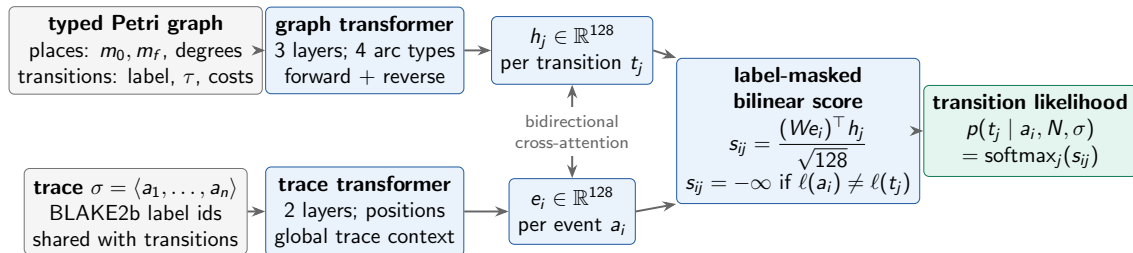
The model learns to **imitate** optimal alignments computed by exact A*:

$$\mathcal{L} = \underbrace{w_m (\mathcal{L}_{\text{sync}} + \mathcal{L}_{\text{log}} + \mathcal{L}_{\text{model}})}_{\text{imitation of moves, } w_m=1.0} + \underbrace{w_c \mathcal{L}_{\text{cost}} + w_b \mathcal{L}_{\text{bound}} + w_\ell \mathcal{L}_{\text{bal}} + w_e \mathcal{L}_{\text{ent}}}_{\text{diagnostic branch, } w \leq 0.03}$$

- $\mathcal{L}_{\text{sync}}$: cross-entropy over transition **identities**, not labels: which concrete transition did the optimum fire? This is what teaches **duplicate-label** resolution
- $\mathcal{L}_{\text{log}} / \mathcal{L}_{\text{model}}$: which events did the optimum **skip**, which transitions did it fire **without a matching event**?
- $\mathcal{L}_{\text{cost}}$: the summed regional cost bounds must match the **optimal cost** δ
- $\mathcal{L}_{\text{bound}}, \mathcal{L}_{\text{bal}}, \mathcal{L}_{\text{ent}}$: regions stay **connected**, **balanced** in size, and **decisive**

The weights keep **move imitation dominant**; the decomposition is a regularized **diagnostic**, not the primary objective.

From Petri Net + Trace to Transition Likelihoods



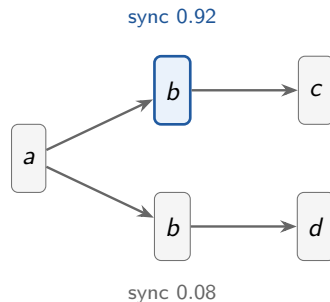
- The net encoder makes each h_j identify a **concrete transition** by initial/final markings, routing, and downstream context—not merely by its label.
- Cross-attention makes both h_j and e_i depend on the **whole net–trace pair**; the same transition can therefore score differently for different trace suffixes.
- The hard mask gives mismatched labels probability zero; among duplicates, the softmax ranks **transition identities**. During decoding, the current marking further restricts the choice to **enabled transitions**.

Where Learning Helps: Duplicate Labels

The decoder reads the trace **one event at a time**, always knowing the **current marking** of the Petri net:

- If exactly **one** enabled transition carries the event's label, the choice is **forced by the Petri net**: no learning needed
- If **several** enabled transitions carry the same label, the learned **synchronous score** decides which concrete transition the event refers to
- If **no** such transition is enabled, the **model-move scores** guide a short search that lets the Petri net catch up
- If even that fails, the event is a deviation: it becomes a **log move**

The candidate is always **replayable**; **optimality** is checked by the exact layer.



For the trace $\langle a, b, c \rangle$, both transitions labeled b are enabled; only the **upper** branch can continue with c , so the learned score selects it.

Training and Validation Data

Exact-labeled synthetic pipeline (imitation of the exact aligner):

- 1 **draw a Petri net** from a curriculum: 80% linear workflows, 20% **duplicate-label** choices differing only in their suffixes
- 2 **perturb** a fitting trace with controlled deviations (rate 0.25: delete, insert, duplicate, swap, append)
- 3 **label** with exact A^* : optimal alignment **with transition identities**, replay-verified

Split	Examples
Training	400
Validation	100
Test	100

Hashed labels $\Rightarrow$ the model learns **structure**, never vocabulary: one checkpoint for any log.

Training: 22 epochs, about 32 s each, **single CPU**.

Evaluation: How

Protocol: all runs on a **single CPU**. LARA always in **fast mode**: the verified neural candidate is scored **as is**, never repaired, so no exact result contaminates the learned metrics. The **exact A*** **optimum** δ is the ground truth for every sample.

- **In-distribution quality:** 100 held-out examples from the training distribution; measures whether every candidate is **legal** (replayable), how often its cost **equals** the optimum δ , and **how far off** it is otherwise
- **Guidance ablation:** identical inputs, but the neural scores are replaced by fixed structural tie-breaks; isolates what **learning** adds over constrained search alone
- **Scaling benchmark:** random block-structured Petri nets with 5 to 40 activities, a family **never seen in training**; locates where the fast path **overtakes exact A*** in wall time
- **Zero-shot real life:** two public logs (building permits, road traffic fines) with Petri nets discovered by the inductive miner; **unseen vocabularies, no fine-tuning**; tests foundation-model **transfer**

Evaluation: Results

In-distribution ($n = 100$)	
legal alignments	100%
certified optimal	88%
mean cost gap	0.38

Ablation: learning adds **+27.7 points** on the **duplicate-label** family (94.4% vs. 66.7%); zero elsewhere.

- **Legality transfers everywhere:**
100% on unseen families and real logs
- Fast path **overtakes exact A*** at about 40 activities ($3.7\times$; 7 to $12\times$ at 80)
- Real discovered Petri net (invisible-heavy): **18.5 ms** vs. 27.4 ms median
- But: optimal-cost rate **drops to about 30%** on large unseen Petri nets

Why Consider LARA in Process Mining?

When exact alignment is the bottleneck, LARA changes the speed–trust trade-off without changing what an alignment means.

learned proposal → **replay check** → **exact certificate or repair**

Faster interaction

A near-fixed CPU forward pass avoids the heavy tail of exact search; the fast path overtakes A* on larger models and already helps on an invisible-heavy discovered net.

Same conformance object

The output is still an event-by-event, transition-identity alignment with a replay-verified cost—usable by existing diagnostics, not an opaque fitness prediction.

Low adoption risk

One vocabulary-free checkpoint applies without per-log retraining; certification can be requested whenever optimality matters, and failures fall back to exact repair.

Practical role: a safe accelerator in front of the exact aligner, **not a replacement for formal guarantees.**

What Can Be Improved: Data

- Training Petri nets are **tiny** (3 to 9 transitions): train on the richer **block-structured generator** (loops, concurrency, invisible routing at scale)
- Quality does **not transfer to large Petri nets** (about 30% optimal at 40 activities): **broader training distributions** are the clear next step
- Only **two real-life logs** tested: validate on **larger log collections**

Takeaway

Learning proposes, exact reasoning certifies: the result is **never less trustworthy** than classical alignment.

- **Legality is unconditional:** every produced alignment can be replayed from the initial to the final marking, and this held in **100%** of the cases on synthetic families, on larger unseen Petri nets, and on real-life logs alike
- **88%** of in-distribution candidates already carry the **exact optimal cost**, so one cheap exact call (about 1 ms) certifies them; the other 12% are **repaired exactly**, never returned wrong
- The learned scores pay off exactly on **duplicate labels**: **94.4%** optimal with guidance vs. 66.7% without; everywhere else, constrained search alone is already strong
- The fast path stays at the **forward-pass floor** (8 to 12 ms per trace) and **overtakes exact A*** at about 40 activities, and already on a real discovered Petri net full of invisible transitions

Checkpoint, scripts, and results: <https://github.com/fit-alessandro-berti/lara-align>

Appendix

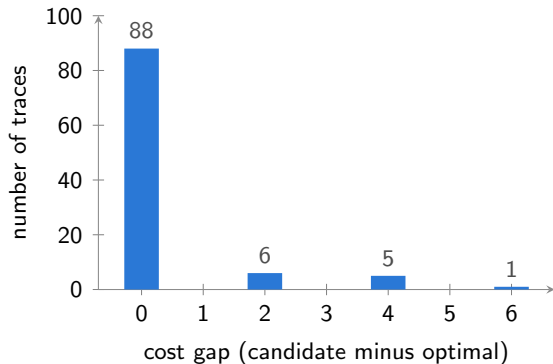
Detailed Evaluation Results

LARA always in fast mode; exact A* optimum as ground truth.

Appendix: In-Distribution Results

Held-out test split ($n = 100$)	
replayable (legal)	100.0%
certified optimal	88.0%
exact transition-sequence match	68.0%
mean / median cost gap	0.38 / 0.00
p95 / max cost gap	4.0 / 6.0

All non-zero gaps are **even**: the signature of the greedy decoder trading k synchronous moves for k model + k log moves, a penalty of exactly $2k$.



Appendix: Guidance Ablation

Identical inputs and decoder; only the neural scores are replaced by fixed structural tie-breaks:

Metric ($n = 100$)	unguided	guided	Δ
replayable	100.0%	100.0%	0.0
equal to optimum cost	83.0%	88.0%	+5.0
exact transition-sequence match	63.0%	68.0%	+5.0
mean cost gap	0.48	0.38	-0.10
sequence family, optimal ($n = 82$)	86.6%	86.6%	0.0
duplicate-label family, optimal ($n = 18$)	66.7%	94.4%	+27.7

- On **sequence** Petri nets every event has at most one matching transition, so guidance changes **nothing**
- The **entire** in-distribution contribution of learning sits in **duplicate-label** identity resolution, exactly as the mechanism predicts

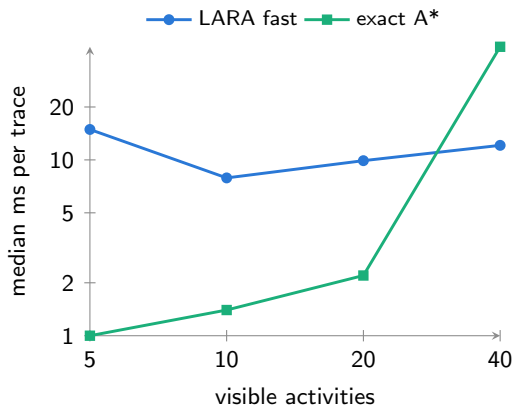
Appendix: Scaling Benchmark

Random block-structured Petri nets, **never seen in training** (10 samples per configuration, 30 s exact timeout):

size	legal	optimal	fast med.	exact med.
5	100%	80%	14.9 ms	1.0 ms
10	100%	80%	7.9 ms	1.4 ms
20	100%	60%	9.9 ms	2.2 ms
40	100%	30%	12.1 ms	44.0 ms

(deviation rate 0.15 shown)

Legality is scale-invariant; the optimal-cost rate **drops** out of distribution, while the fast path stays at the **forward-pass floor**.



Appendix: Zero-Shot Results on Real-Life Logs

Same checkpoint, **no fine-tuning**; Petri nets discovered by the inductive miner (noise 0.0 = perfectly fitting, 0.5 = filtered):

Log	Noise	Legal	Optimal (trace-wtd.)	Max gap	Median LARA / exact
road traffic	0.0	100%	100.0%	0	8.0 / 1.7 ms
road traffic	0.5	100%	94.0%	3	7.9 / 1.3 ms
receipt	0.0	100%	82.6%	5	18.5 / 27.4 ms
receipt	0.5	100%	74.6%	7	8.9 / 8.1 ms

- **Legality transfers perfectly** to unseen vocabularies and invisible-heavy discovered models (47 invisible vs. 27 visible transitions on the fitting receipt net)
- Optimality is strong on **frequent** behavior; misses concentrate in **rare, long variants** and remain safe upper bounds
- On the largest discovered Petri net, the fast path **beats exact A*** on a real log (18.5 vs. 27.4 ms median)